

MANUAL DE USUARIO

Hermes Stack

Tu asistente IA personal

Cómo usar tu sistema con máxima eficiencia y calidad

Flujos de trabajo · Rutinas diarias · Comandos esenciales
Resolución de problemas · Mejores prácticas comprobadas

Versión: 1.0 — Mayo 2026

Para: Erik José Hernández

Sistema: Hermes Stack v3.0

Dominio: el80.space

VPS: Hostinger 4vCPU / 8GB

Compilado el 28 de mayo de 2026 con XeLaTeX
github.com/erikjosehrnndz-crypto/hermes-stack

Índice de contenidos

Índice

1. Bienvenido a tu sistema	4
1.1. Qué es Hermes Stack	4
1.2. Qué vas a encontrar en este manual	4
1.3. Las 3 reglas de oro del sistema	4
2. Mapa mental del sistema	5
2.1. Los servicios que tienes que conocer	5
2.2. Los archivos que tienes que conocer	5
2.3. Diagrama del flujo de voz	5
2.4. Cómo identificar de un vistazo si todo está bien	5
3. Rutina diaria recomendada	6
3.1. Apertura de jornada (5 minutos)	6
3.2. Durante la jornada	6
3.3. Cierre de jornada (5 minutos)	7
4. Flujos de trabajo principales	8
4.1. Flujo A — Conversar con Hermes (uso normal)	8
4.2. Flujo B — Añadir una habilidad nueva a Hermes	8
4.3. Flujo C — Modificar el frontend (Next.js)	8
4.4. Flujo D — Investigar un tema con sub-agentes	9
4.5. Flujo E — Deploy nueva versión del agente	9
4.6. Flujo F — Generar documento profesional (LaTeX)	9
5. Resolución de problemas	10
5.1. Tabla de diagnóstico rápido	10
5.2. Si un contenedor está unhealthy	10
5.3. Si el deploy CI/CD hizo rollback automático	10
5.4. Si LiteLLM reporta modelo caído	10
5.5. Si los logs llenan el disco	11
5.6. Si Claude Code se queda atascado	11
6. Personalizar y extender el sistema	12
6.1. Editar las instrucciones de Claude Code (CLAUDE.md)	12
6.2. Añadir una memoria en el sistema de auto-memoria	12
6.3. Cambiar el modelo LLM por defecto	12
6.4. Cambiar la voz de Hermes	12
6.5. Añadir un subdominio nuevo (servicio público)	12
6.6. Activar/desactivar Brain Phase 6 (memoria conversacional)	13
7. Mantenimiento semanal	14
7.1. Lista de chequeo (15-20 minutos, viernes recomendado)	14
7.2. Limpieza profunda (mensual)	14
7.3. Métricas a revisar semanalmente	14

8. Optimización y mejora continua	15
8.1. Reducir latencia (las 5 palancas)	15
8.2. Reducir costo	15
8.3. Mejorar calidad de las respuestas	15
8.4. Aumentar privacidad	16
8.5. Mejorar resiliencia	16
9. Seguridad y buenas prácticas	17
9.1. Lo que NUNCA debes hacer	17
9.2. Lo que SIEMPRE debes hacer	17
9.3. Gestión de secretos — .env	17
9.4. Si un servicio se ve comprometido	18
10. Visión a futuro — qué viene	19
10.1 Roadmap personal recomendado	19
10.2 Conceptos a estudiar (para entender mejor el stack)	19
10.3 Cómo pensar en escalar (cuándo)	19
11. Referencia rápida de comandos	20
11.1 Top 20 comandos del día a día	20
11.2 Variables de entorno críticas (.env)	20
11.3 URLs útiles	21
11.4 Atajos del flujo Hermes	21
11.5 Cuándo escalar al usuario (a ti mismo) vs Claude	21
11.6 Documentos relacionados	21

1. Bienvenido a tu sistema

1.1. Qué es Hermes Stack

Hermes es **tu asistente IA personal autohospedado**. Vive en un servidor que tú controlas (el80.space, en Hostinger), responde por voz y por texto, recuerda lo que hablan, y se ejecuta sin enviar tu información a ningún servicio comercial que no decidas explícitamente.

A diferencia de un chatbot comercial (ChatGPT, Gemini, Claude.ai), Hermes:

- **Te pertenece.** Vive en tu VPS, en tu repositorio Git, con tus claves.
- **Tiene memoria persistente.** Brain guarda eventos, contexto y embeddings.
- **Es extensible.** Cada nueva habilidad se añade como un skill MCP.
- **Cuesta poco.** \$37/mes total entre VPS + LLMs + voz.
- **No depende de un solo proveedor.** LiteLLM enruta entre 5 modelos.

1.2. Qué vas a encontrar en este manual

Este manual está organizado por **intención**: qué quieres hacer, cómo se hace, qué evitar. No es una referencia técnica exhaustiva (eso es la `ficha_tecnica_hermes.pdf`); es la guía operativa que usas el día a día.

Sección	Cuándo leerla
1-2	Antes de empezar — orientación general
3	Cada mañana — rutina diaria recomendada
4	Cuando vas a hablar/escribir con Hermes
5	Cuando algo no funciona
6	Cuando quieres añadir o cambiar algo
7	Cuando vas a cerrar la jornada
8	Una vez por semana — mantenimiento
9-10	Una vez al mes — optimización y revisión
11	Referencia rápida de comandos

1.3. Las 3 reglas de oro del sistema

● PASO 1 — Verificar antes de declarar

“Debería funcionar” no es una verificación. Antes de creer que algo está bien (un deploy, un cambio, una respuesta), ejecuta el comando que lo prueba: `make doctor`, `curl al health`, o una pregunta real a Hermes.

● PASO 2 — Commitear por tarea, no por sesión

Si terminaste una tarea, haz commit. No acumules cambios de 5 tareas en un solo commit al final del día. Si tu sesión cae por context limit, pierdes el trabajo no commiteado.

● PASO 3 — Una tarea activa por sesión

Termina lo que empezaste antes de abrir otra cosa. Si surge algo nuevo durante el trabajo, anótalo en `PENDIENTES.md` pero no lo empieces. Cambiar de contexto deja 3 cosas al 70 % en lugar de 1 al 100 %.

2. Mapa mental del sistema

2.1. Los servicios que tienes que conocer

De los 15 contenedores que corren en tu VPS, hay 5 que vas a tocar habitualmente. Los otros 10 funcionan en background.

Servicio	URL	Para qué lo abres
hermes-website	docs.el80.space	Hablar con Hermes, ver el tree del re-po
hermes-agent	hermes.el80.space	Endpoint API, health check
grafana	grafana.el80.space	Ver métricas y latencia en tiempo real
filebrowser	files.el80.space	Subir/bajar archivos al VPS sin SSH
brain	brain.el80.space	Memoria de Hermes (cliente MCP)

2.2. Los archivos que tienes que conocer

Archivo	Qué contiene
/root/CLAUDE.md	Instrucciones operativas para Claude Code en este proyecto
/root/PENDIENTES.md	Tu lista de tareas activas (estado actual + próximo paso)
/root/SESSION_HANDOFF.md	Si existe: notas para retomar la sesión anterior
/root/Makefile	Targets de verificación: make doctor, make check
/root/docker-compose.yml	Definición de los 15 contenedores del stack
/root/.env	Secretos y tokens — NUNCA al repositorio

2.3. Diagrama del flujo de voz



Total 500 ms (SLO cumplido)

2.4. Cómo identificar de un vistazo si todo está bien

Tres señales que indican que el sistema está saludable:

1. docker compose ps muestra todos los servicios con estado Up X (healthy).
2. docs.el80.space carga sin errores y permite enviar una pregunta.
3. grafana.el80.space muestra métricas recientes (no plano hace 30 min).

Si las tres se cumplen → el sistema está al 100 %. Si alguna falla → ver la Sección 5 (Resolución de problemas).

3. Rutina diaria recomendada

3.1. Apertura de jornada (5 minutos)

Cada mañana, antes de empezar trabajo serio, ejecuta este chequeo. Te ahorra horas de debugging si algo se rompió durante la noche.

● PASO 1 — Conectar al VPS

Conéctate por SSH (`ssh root@el80.space` o el alias que tengas configurado).

● PASO 2 — Estado del Git

Revisa el estado del repo. Si hay cambios sin commit de la noche anterior: revísalos. O bien commit, o bien descarta — **no empieces trabajo nuevo encima de cambios sin clasificar**.

```
cd /root
git status
git log --oneline -5
```

● PASO 3 — Health check completo

Ejecuta `make doctor`. Espera ver OK en cada paso (repo, docker, health, lint, harness, pendientes). Si hay un fallo, lee la sección 5.

```
make doctor
```

● PASO 4 — Revisar PENDIENTES

Identifica la tarea activa para hoy. **Una tarea por sesión**.

```
head -30 /root/PENDIENTES.md
```

● PASO 5 — Si existe SESSION_HANDOFF.md

La sesión anterior dejó instrucciones específicas. Léelas antes de actuar.

```
cat /root/SESSION_HANDOFF.md
```

3.2. Durante la jornada

Trabajando con Claude Code (interactive)

Abre Claude Code en `/root`. El sistema `CLAUDE.md` se carga automáticamente — Claude conoce tus reglas operativas.

● BUENA PRÁCTICA: No micro-gestiones a Claude

Si quieres una tarea no trivial, descríbela completa: objetivo, restricciones, archivos involucrados. Claude planificará internamente (ver "Planificación interna implícita.^{en} CLAUDE.md). Interrumpir cada 2 pasos consume tokens y rompe el flujo.

● BUENA PRÁCTICA: Pide commits explícitos

Claude no commit automáticamente. Cuando termina una tarea lógica, dile: `commitea esto`. O configura: `commitea` cada vez que termines una tarea.

Hablando con Hermes (voz/texto)

Abre <https://docs.el80.space> en cualquier navegador. La interfaz tiene un input de texto y, si concedes permisos, captura de voz.

● TIP: Frase corta = mejor latencia

El cache de LiteLLM ahorra 33 % de queries cuando repites frases similares. Frases cortas también significan menos tokens STT → menos latencia.

● CUIDADO: Privacidad de voz

Tu audio va a Whisper (local, OK) y a ElevenLabs (cloud, vé el texto). Si lo que dices es muy sensible, escríbelo en texto en lugar de hablarlo.

Consultando memoria de Brain

Brain recuerda lo que has hablado. Puedes preguntarle:

- *"¿De qué hablamos la semana pasada sobre los frenos?"*
- *"¿Qué decisiones tomé sobre la migración a Next.js 15?"*
- *Resumen de mis pendientes de hace 3 días"*

Brain usa retrieval híbrido (RRF) entre vector search + graph hops. Cuanto más concreto el query, mejor el resultado.

3.3. Cierre de jornada (5 minutos)

● PASO 1 — Actualizar PENDIENTES.md

Marca lo que terminaste como done y añade evidencia (commit hash, output, URL). Si quedó algo a medias, deja claro el próximo paso.

● PASO 2 — Verificar que el stack sigue sano

Ejecuta `make health-check` (rápido, solo `/health`).

● PASO 3 — Commit final del día

Asegúrate de no dejar cambios sin commit. Si hay trabajo incompleto, commit como `wip:` con descripción.

● PASO 4 — Si dejas trabajo a medias

Copia la plantilla de handoff y rellena la última acción + próximo paso recomendado.

```
cp .claude/templates/session-handoff.md /root/SESSION_HANDOFF.md
```

● PASO 5 — Backup (cada viernes)

Ejecuta `make backup` y confirma con `make backup-list`.

4. Flujos de trabajo principales

4.1. Flujo A — Conversar con Hermes (uso normal)

Objetivo: hacer una pregunta, recibir respuesta de voz/texto.

1. Abrir docs.el80.space.
2. Si es por voz: pulsar el botón de micrófono, hablar, soltar.
3. Esperar respuesta (objetivo: <500ms total).
4. Si la respuesta es relevante, Brain la guarda automáticamente.
5. Para profundizar: pide *.explica más sobre X* en la siguiente vuelta.

● **BUENA PRÁCTICA: Aprovecha la memoria**

Si vuelves al mismo tema al día siguiente, di *¿continuamos con lo de ayer sobre X*. Brain recupera el contexto vía retrieval semántico y Hermes lo trae al prompt.

4.2. Flujo B — Añadir una habilidad nueva a Hermes

Objetivo: que Hermes pueda hacer algo que antes no hacía (ej. consultar el clima, leer noticias).

1. Decidir si la habilidad es **interna** (skill Python en /root/hermes/skills/) o **externa** (MCP server independiente).
2. Para skill Python: añadir archivo my_skill.py con función async. Importarla en hermes/agent.py.
3. Para MCP server: crear contenedor nuevo en docker-compose.yml expuesto en localhost. Apuntar Claude Code/Hermes a la URL MCP.
4. Test local antes de deploy: curl al endpoint.
5. Commit + push → CI/CD aplica el cambio en el VPS.
6. Verificar con make doctor.

● **CUIDADO: Skills no triviales requieren plan**

Para añadir una skill que toca varios archivos, primero pide a Claude Code que diseñe el plan (/plan). Aprueba el plan antes de empezar. Esto evita refactors a media implementación.

4.3. Flujo C — Modificar el frontend (Next.js)

Objetivo: cambiar la interfaz web de docs.el80.space.

```
cd /root/website
npm install           # solo si cambiaron deps
# editar src/App.tsx o componentes en src/components/
npm run dev           # localhost:3001 para preview
npm run build         # VERIFICAR que compila sin error
git add website/<archivos>
git commit -m "feat(web): ..."
git push              # CI/CD deploya
```

● **CUIDADO: Build antes de commit, siempre**

Si commiteas frontend sin verificar npm run build, el CI/CD falla y hace rollback. Pierdes 5-10 minutos. Verifica antes — toma 30 segundos.

4.4. Flujo D — Investigar un tema con sub-agentes

Objetivo: pedir a Claude Code que investigue algo profundo (paper, librería, decisión arquitectónica).

1. Verifica que estás en **Opus 4.7** (/model). El orquestador raíz necesita el mejor modelo.
2. Describe el objetivo con suficiente contexto: dominio, restricciones, output esperado.
3. Si la investigación es exhaustiva, di /plan y aprueba el swarm antes de ejecutar.
4. Para tareas con SOTA: añade máximo esfuerzo o state of the art — Claude lanzará WebSearch antes de diseñar.
5. Espera sin interrumpir. Los sub-agentes corren en background.
6. Verifica que el output incluye archivos en /tmp/ (checkpointing).

● **BUENA PRÁCTICA: Confía en el orquestador**

Cuando lanzas un swarm, no preguntes "¿cómo va?" cada 2 minutos. Los sub-agentes están corriendo. Te notificarán al terminar. Interrumpir aborta el plan.

4.5. Flujo E — Deploy nueva versión del agente

Objetivo: cambiar lógica del agente Python sin downtime.

```
cd /root
# editar hermes/agent.py
docker compose build hermes hermes-worker
docker compose up -d hermes hermes-worker
curl http://127.0.0.1:8080/health # verificar
```

● **BUENA PRÁCTICA: Servicios que comparten Dockerfile se rebuildan juntos**

hermes y hermes-worker (y brain/brain-worker) comparten imagen. Reconstruir solo uno deja el otro con el código antiguo. Reconstruir AMBOS.

4.6. Flujo F — Generar documento profesional (LaTeX)

Objetivo: crear un PDF profesional para un informe, manual, ficha.

```
cd /root/docs
# crear o editar mi_doc.tex
xelatex -interaction=nonstopmode mi_doc.tex
xelatex -interaction=nonstopmode mi_doc.tex # TOC + labels
xelatex -interaction=nonstopmode mi_doc.tex # outlines
grep "^!" mi_doc.log | sort -u # debe salir vacio
pdfinfo mi_doc.pdf | grep Pages # confirmar paginas
```

● **CUIDADO: Babel español + TikZ exige orden específico**

Siempre \usetikzlibrary{babel,arrows.meta,...} con babel PRIMERO. Si no, los > fallan en flechas.

5. Resolución de problemas

5.1. Tabla de diagnóstico rápido

Síntoma	Causa probable	Acción
docs.el80.space no carga	website caído o Caddy	<code>docker compose restart hermes-website caddy</code>
Hermes no responde voz	whisper-stt o litellm caído	<code>docker compose logs --tail=50 whisper-stt litellm</code>
Latencia >1 segundo	cache miss en LiteLLM	Verifica modelo en uso (/metrics/); evita modelos lentos
Health check falla	env var no cargada	<code>source /root/.env; verifica LITELLM_MASTER_KEY</code>
Build Next.js falla	deps desincronizadas	<code>cd website && rm -rf node_modules && npm install</code>
Git push pide credenciales	token no inyectado	Usar patrón HTTPS con GH_TOKEN (CLAUDE.md §Git)
Kuzu lock error	dos procesos abren write	Solo brain-worker escribe; brain lee read_only
CI/CD hizo rollback	health check fallido post-deploy	<code>git log -1</code> ; revisar diff que rompió

5.2. Si un contenedor está unhealthy

```

docker compose ps           # ver cual no esta Up
docker compose logs --tail=100 <name> # ver el error
docker inspect <name> | grep -A 5 Health
docker compose restart <name>      # reinicio limpio
docker compose ps               # confirmar Up (healthy)

```

5.3. Si el deploy CI/CD hizo rollback automático

1. GitHub Actions detectó health check failed y ejecutó `git reset --hard HEAD 1`.
2. El VPS volvió al commit anterior.
3. Tu rama main en GitHub sigue con el cambio que rompió.
4. Para corregir: o bien `commit fix` encima, o bien `git revert HEAD` y push.

● CUIDADO: Nunca force-pushees main

Si haces `git push --force main`, sobrescribes el historial y rompes el flujo CI/CD para todos los siguientes deploys. Si necesitas revertir, usa `git revert` (commit que deshace).

5.4. Si LiteLLM reporta modelo caído

```
source /root/.env
curl -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
  http://127.0.0.1:4000/v1/models
# si un modelo no aparece -> esta deshabilitado por LiteLLM
curl -H "Authorization: Bearer $LITELLM_MASTER_KEY" \
  http://127.0.0.1:4000/health
```

Causas más comunes:

- Cuota del proveedor agotada → revisar billing en OpenRouter/Anthropic.
- API key rotada y no actualizada en .env.
- Modelo deshabilitado por LiteLLM tras varios fallos consecutivos.

5.5. Si los logs llenan el disco

```
df -h                                # uso del disco
docker system df                     # uso de Docker
docker compose logs --tail=10 hermes # ver volumen logs
# si el problema es logs:
docker system prune -f               # limpia logs viejos
# si el problema es DBs:
du -sh /root/brain_*/                # tamanos brain
```

5.6. Si Claude Code se queda atascado

1. Pulsa Esc para interrumpir el tool actual.
2. Si no responde, abre otra terminal y haz `ps -ef | grep claude`.
3. Si está realmente colgado: cerrar sesión y reabrir.
4. Si los hooks de progreso quedaron stale: `rm -f /tmp/claude_progress`.

6. Personalizar y extender el sistema

6.1. Editar las instrucciones de Claude Code (CLAUDE.md)

CLAUDE.md es donde defines cómo quieres que Claude Code se comporte. Cada vez que abras una sesión, esas instrucciones se cargan automáticamente.

● **CUIDADO: Sólo el agente principal edita CLAUDE.md**

Los sub-agentes que intenten editarlo reciben un security warning. Si necesitas un cambio, hazlo desde tu sesión principal o pídeselo explícitamente a Claude.

● **BUENA PRÁCTICA: Mantén CLAUDE.md <40 KB**

Por encima de ese tamaño Claude lanza un warning de performance. Si crece: comprime prosa explicativa, no elimines reglas operativas. `wc -c /root/CLAUDE.md` para verificar.

6.2. Añadir una memoria en el sistema de auto-memoria

Si quieres que Claude recuerde algo entre sesiones (preferencia, decisión, contexto):

```
# Las memorias viven en:
ls /root/.claude/projects/-root/memory/
# Indice: MEMORY.md
# Detalles: feedback_*.md, project_*.md, user_*.md, reference_*.md
```

Para añadir una memoria nueva, dile a Claude: *recuerda que prefiero X sobre Y, porque Z*". Se guarda como archivo en la carpeta y se enlaza desde MEMORY.md.

6.3. Cambiar el modelo LLM por defecto

Edita `config/litellm.yaml` y modifica el orden en `model_list` o cambia el `routing_strategy`. Reinicia LiteLLM:

```
docker compose up -d --force-recreate litellm
```

6.4. Cambiar la voz de Hermes

ElevenLabs ofrece muchas voces. Edita la variable `ELEVENLABS_VOICE_ID` en `.env` y reinicia el agente:

```
nano /root/.env # actualizar ELEVENLABS_VOICE_ID
docker compose up -d hermes
```

● **TIP: Probar voces antes de cambiar**

Entra a elevenlabs.io/voice-library, escucha samples, copia el voice ID. Voces más populares para español: "Sarah", "Adam", "Bella", "Antoni".

6.5. Añadir un subdominio nuevo (servicio público)

Para exponer un nuevo servicio bajo `nuevo.el80.space`:

1. Añadir el subdominio en Hostinger DNS apuntando a la IP del VPS.

2. Añadir el bloque correspondiente en Caddyfile:

```
nuevo.el80.space {  
    reverse_proxy 127.0.0.1:XXXX  
}
```

3. Recargar Caddy: `docker compose restart caddy`.

4. Verificar TLS: `curl -I https://nuevo.el80.space` (debe 200 o 401).

6.6. Activar/desactivar Brain Phase 6 (memoria conversacional)

Brain Phase 6 captura cada turn (user + assistant) y crea embeddings asíncronos vía RQ worker. Está activado por defecto. Para revisar:

```
docker compose logs --tail=30 brain-worker  
# debe mostrar 'job consolidate done' periodicamente
```

7. Mantenimiento semanal

7.1. Lista de chequeo (15-20 minutos, viernes recomendado)

1. **Backup completo:** `make backup` y verificar `make backup-list`.
2. **Revisión de PENDIENTES:** ¿hay tareas `⏏` warning que se están enfriando? Re-priorizar.
3. **Limpieza de `settings.local.json`:** si tiene >20 reglas → mover las relevantes a `settings.json` y limpiar.

```
cat ~/.claude/settings.local.json | jq '.permissions.allow | length'
echo '{}' > ~/.claude/settings.local.json    # solo si la migracion ya esta hecha
```

4. **Limpieza de `/tmp`:** archivos de sub-agentes que no se integraron.

```
ls -la /tmp/claude_progress /tmp/*.md 2>/dev/null
# revisar y eliminar lo no necesario
```

5. **Métricas Grafana:** revisar dashboard de Hermes — ¿latencia p95 dentro de SLO?
6. **Logs de errores:** `docker compose logs --tail=200 hermes | grep -i error`.
7. **Espacio en disco:** `df -h`; si >80 % usado, limpieza.
8. **Dependabot:** revisar PRs de security updates en GitHub.
9. **Renovación TLS:** confirmar que Caddy renovó certificados (auto).

7.2. Limpieza profunda (mensual)

```
docker system prune -a -f --volumes    # PELIGROSO: solo si backups recientes
docker compose pull                    # actualizar imagenes base
docker compose up -d --force-recreate  # recrear con imagenes nuevas
make doctor                            # verificacion completa
```

● CUIDADO: No ejecutes `docker system prune --volumes` sin backup

Eso borra volúmenes Docker no referenciados. Si por alguna razón un volumen activo no aparece como `in use`.^{en} ese momento (servicio caído), se pierden datos. Backup primero.

7.3. Métricas a revisar semanalmente

Métrica	Objetivo	Alerta si
Latencia E2E p50	<500 ms	>700 ms
Latencia E2E p95	<800 ms	>1200 ms
Cache hit rate LiteLLM	>25 %	<15 %
CPU promedio del VPS	<30 %	>60 %
RAM uso	<70 %	>85 %
Uptime hermes-agent	100 %	<99.5 %
Coste mensual LLM	<\$15	>\$25

8. Optimización y mejora continua

8.1. Reducir latencia (las 5 palancas)

1. **Cache de LiteLLM activo:** verifica que `cache: True` en `config/litellm.yaml`.
2. **ClientSession aiohttp compartida:** ya implementado en `hermes/agent.py` (no crear sesión por request).
3. **Modelo más rápido para queries simples:** Gemini Flash < GPT-4o-mini < Claude Sonnet en latencia.
4. **Whisper modelo base:** no subir a small o medium a menos que veas errores recurrentes.
5. **ElevenLabs Flash v2.5:** el modelo Multilingual v2 es más bonito pero suma 200 ms.

8.2. Reducir costo

- **Routing por latencia:** ya envía a Gemini Flash por defecto (\$0.075/M tokens).
- **Cache redis activo:** ahorra 33 % de queries (datos reales).
- **TTLCache para dedup de voz:** elimina queries duplicadas por eco del micrófono.
- **Truncar contexto largo:** no enviar más de los últimos N turns al prompt (config Hermes).

● **TIP: Modelos caros solo cuando hacen falta**

Claude Sonnet 4.6 cuesta 40× más que Gemini Flash. Úsalo cuando necesites razonamiento profundo (decisiones técnicas, análisis), no para charla casual.

8.3. Mejorar calidad de las respuestas

System prompt optimizado

Edita el prompt base de Hermes en `hermes/agent.py`. Buen prompt:

- **Rol claro:** ".Eres Hermes, asistente personal de Erik..."
- **Estilo:** Respondes en español, conciso, sin emojis."
- **Restricciones:** "Si no sabes algo, dilo. No inventes citas o datos."
- **Memoria:** "Úsa la memoria de Brain cuando sea relevante."

Calidad de retrieval (Brain)

Si Brain devuelve resultados poco relevantes:

1. Verifica que el modelo embeddings esté cargado (`docker compose logs brain | grep embed`).
2. Re-indexa: `docker compose exec brain-worker python3 -m brain.reindex`.
3. Considera ajustar el cross-encoder reranker (puede ayudar para queries ambiguas).

8.4. Aumentar privacidad

- **LLM local con GPU:** mover queries sensibles a Llama 3.3 70B local (cuando montes GPU).
- **TTS local:** fallback a Coqui XTTS-v2 con GPU para textos sensibles.
- **Filtrar logs:** no loguear el contenido completo de mensajes — solo metadata.

8.5. Mejorar resiliencia

- **Fallbacks LiteLLM:** configurar chain Gemini Flash → GPT-4o-mini → Llama 3.1.
- **Healthchecks agresivos:** interval 30s, autoheal active.
- **Backups remotos:** además de make backup local, sincronizar a S3 o similar.
- **Monitoreo externo:** servicio como UptimeRobot que pingue /health cada 5 min.

9. Seguridad y buenas prácticas

9.1. Lo que NUNCA debes hacer

● **CUIDADO: Nunca commit secretos al repositorio**

API keys, tokens, contraseñas. Si ocurre por error: rotar la clave inmediatamente (no basta con `git rm`). El historial Git público es indexado por bots en horas.

● **CUIDADO: Nunca force-push a main**

Sobreescribe historial, rompe CI/CD, puede perder trabajo de otros. Si necesitas deshacer: `git revert`.

● **CUIDADO: Nunca skip pre-commit hooks (-no-verify)**

Los hooks ejecutan tests, lint, build. Si los saltas, commiteas código roto. Si el hook falla: investiga la causa, arregla, re-commit.

● **CUIDADO: Nunca expongas servicios sin auth**

Cualquier puerto que abras al público (más allá de localhost) requiere autenticación. Sin auth = vector de ataque. Caddy puede añadir basic auth en pocas líneas.

● **CUIDADO: Nunca apliques cambios rápidos.^{en} producción sin testing**

Editar archivos directamente en el VPS y reiniciar sin commit deja el VPS desincronizado del repo. Cualquier CD posterior sobreescribe tu cambio. Workflow correcto: commit → push → CD aplica.

9.2. Lo que SIEMPRE debes hacer

● **BUENA PRÁCTICA: Siempre verifica con `make doctor` antes de declarar algo "listo"**

El comando ejecuta 6 chequeos en 30 segundos. Te ahorra 30 minutos de debugging por suponer que todo está bien.

● **BUENA PRÁCTICA: Siempre commit por tarea lógica**

No acumules cambios. Si terminaste algo discreto, commit. La granularidad de tu historial Git es tu seguro contra accidentes.

● **BUENA PRÁCTICA: Siempre lee el archivo completo antes de editarlo**

No Edit al final del archivo sin leer las últimas líneas. Puede haber contenido de sesiones anteriores que invalida el contexto.

● **BUENA PRÁCTICA: Siempre rota tokens cada 90 días**

API keys de OpenRouter, Anthropic, ElevenLabs. Calendario: 1 de cada cuarto. Permite detectar accesos comprometidos antes de que sean dañinos.

9.3. Gestión de secretos — .env

`/root/.env` contiene todos los tokens críticos. Reglas:

1. Permisos 600 (solo root puede leer): `chmod 600 /root/.env`.
2. Nunca `cat .env` en una sesión compartida — alguien puede ver tu pantalla.
3. Backup encriptado del `.env` en un sitio diferente al VPS (password manager).
4. Si sospechas compromiso: rota TODAS las claves antes de investigar el incidente.

9.4. Si un servicio se ve comprometido

1. Tirar el servicio: `docker compose stop <name>`.
2. Aislar el VPS de la red pública: cerrar puertos en el firewall Hostinger.
3. Rotar TODAS las API keys.
4. Auditar logs: `docker compose logs --since 7d <name>`.
5. Restaurar desde backup previo confirmado limpio.
6. Investigar el vector de entrada antes de reabrir.

10. Visión a futuro — qué viene

10.1. Roadmap personal recomendado

Cuándo	Mejora	Impacto
Próximas 2 semanas	Migrar Whisper a faster-whisper	Alto (-145ms)
Próximo mes	Añadir Loki para logs centralizados	Medio
Próximo mes	Reauth rclone para backups remotos	Medio (resiliencia)
Q3 2026	Mesh Tailscale con Android + DO Droplet	Alto (movilidad)
Q4 2026	Migrar Redis → Valkey	Bajo (licencia)
Q4 2026	Considerar Next.js 15	Bajo
Q1 2027	OpenTelemetry traces	Alto (observabilidad)
H2 2027	GPU local + Llama 3.3 70B	Alto (privacidad+costo)
H2 2027	Memoria episódica (Letta-style)	Muy alto

10.2. Conceptos a estudiar (para entender mejor el stack)

1. **Model Context Protocol (MCP):** cómo funcionan los servidores stdio vs HTTP. Lee la spec de Anthropic.
2. **Retrieval híbrido (RRF):** reciprocal rank fusion entre dense + sparse + cross-encoder.
3. **HNSW:** algoritmo de búsqueda aproximada que usa LanceDB.
4. **Cypher (Kuzu):** lenguaje de queries para grafos. La parte de MATCH (a)-[r]->(b) es 80 % de los queries.
5. **Caddy y ACME:** cómo se renuevan TLS automáticamente (RFC 8555).
6. **Docker networking:** diferencia entre bridge default, bridge custom y host networking.

10.3. Cómo pensar en escalar (cuándo)

El stack actual escala vertical hasta llenar el VPS de 4 vCPU / 8 GB. Eso significa:

- 50 conversaciones de voz simultáneas (limitadas por LiteLLM RPS).
- 10M de vectores en LanceDB (antes de degradar query <50ms).
- 500k nodos en Kuzu (antes de evaluar Neo4j).
- 100 GB de almacenamiento Brain (vault + events + DB files).

Por debajo de esos límites, escalar horizontal (más VPS, K8s) es overkill. Por encima, considera:

1. Subir el tier del VPS Hostinger (8 vCPU / 16 GB) — solución simple.
2. Mesh Tailscale con un segundo nodo para LLM heavy queries.
3. GPU dedicada (RunPod on-demand) para Whisper y/o vLLM.

11. Referencia rápida de comandos

11.1. Top 20 comandos del día a día

```
# Estado y health
make doctor # check completo
make health-check # solo health endpoints
make status # docker compose ps
docker compose logs --tail=50 <svc> # logs recientes
curl -f http://127.0.0.1:8080/health # hermes health

# Git
git status
git log --oneline -10
git add <archivo> && git commit -m "..."
gh pr create --title "..." --body "..."

# Frontend
cd /root/website && npm run build # ANTES de commit
cd /root/website && npm run dev # preview local

# LaTeX
cd /root/docs && xelatex mi_doc.tex # 3 pasadas para TOC

# Docker
docker compose ps
docker compose up -d <svc> # arrancar
docker compose restart <svc> # reinicio limpio
docker compose build <svc> # rebuild imagen
docker compose logs -f <svc> # follow logs

# Backup
make backup # backup completo
make backup-list # listar backups
```

11.2. Variables de entorno críticas (.env)

Variable	Para qué
LITELLM_MASTER_KEY	Auth interna LiteLLM
OPENROUTER_API_KEY	LLMs (Gemini, Llama, GPT)
ANTHROPIC_API_KEY	Claude Sonnet 4.6
ELEVENLABS_API_KEY	TTS
ELEVENLABS_VOICE_ID	Voz de Hermes
BRAIN_API_TOKEN	Auth Brain MCP
HOSTINGER_API_KEY	DNS automation
GRAFANA_ADMIN_PASSWORD	Login Grafana

11.3. URLs útiles

Servicio	URL
Hermes Web	https://docs.el80.space
Grafana	https://grafana.el80.space
Filebrowser	https://files.el80.space
Brain MCP	https://brain.el80.space/mcp/
LiteLLM Health	https://litellm.el80.space/health
Repo GitHub	github.com/erikjosehrndz-crypto/hermes-stack
GitHub Actions	github.com/erikjosehrndz-crypto/hermes-stack/actions
Hostinger Panel	hpanel.hostinger.com
OpenRouter Dashboard	openrouter.ai/keys
ElevenLabs Dashboard	elevenlabs.io/app

11.4. Atajos del flujo Hermes

Quiero	Decir/Escribir a Hermes
Recordar algo	recuerda que [hecho]"
Buscar en memoria	"¿qué recuerdas de [tema]?"
Resumir reciente	resumen de los últimos N días"
Olvidar algo	.olvida lo que sabías de [tema]"
Cambiar tema	¿cambiamos a [nuevo tema]"
Pedir contexto previo	¿continúa con lo de ayer sobre X"
Salir limpio	"guarda estado y nos vemos"

11.5. Cuándo escalar al usuario (a ti mismo) vs Claude

Decisión técnica	Decisión personal
Claude puede decidir: refactor menor, fix de bug, elegir librería entre 2 opciones del SOTA, lint, format.	Tú decides: cambios arquitectónicos, gastos >\$10/mes, exposición pública de un servicio, migraciones disruptivas.

11.6. Documentos relacionados

Documento	Cuándo abrirlo
ficha_tecnica_hermes.pdf	Inventario técnico exhaustivo (55 pp)
estado_del_arte_hermes.pdf	Comparativa SOTA por categoría (47 pp)
Hermes_Stack_Blueprint.pdf	Arquitectura definitiva v2 (52 pp)
Roadmap_Tecnico_Definitivo_v2.pdf	Roadmap consolidado
informe-practicas-ia.pdf	Análisis crítico de patrones
CLAUDE.md (raíz del repo)	Reglas operativas Claude Code
PENDIENTES.md (raíz del repo)	Tareas activas

Fin del manual

Hermes Stack — Manual de Usuario v1.0

Compilado el 28 de mayo de 2026 con XeLaTeX · DejaVu Fonts

*Si encuentras algo desactualizado o pasos que no funcionan:
edita este documento y commitea. La documentación se mantiene
con el mismo cuidado que el código.*